

Tener en cuenta para la guía 10

El proyecto quedó en un punto estable para seguir avanzando porque ya se consolidó la materialización de la guía 09 sobre la base de las guías 06, 07 y 08, sin rehacer el proyecto desde cero. La estructura actual ya tiene una línea de trabajo principal alineada con la arquitectura pedida en la segunda entrega: domain, application.usecase e infrastructure.

Se completó y consolidó la capa de aplicación basada en casos de uso. Además de los casos de uso creados en la guía 06, se agregaron y conectaron nuevos casos para cubrir los flujos que la API ya expone, incluyendo clasificación, priorización, obtención y listados de solicitudes, además de los casos de uso de usuarios para creación, consulta, listado, activación y desactivación. Con esto, la lógica orquestadora ya quedó centrada en application.usecase, que es la línea correcta para continuar con las siguientes guías.

También se completó la frontera de API REST para cumplir mejor con lo exigido por las guías 07 y 08 y por la entrega 02. Se mantuvieron y aprovecharon los DTOs record de request ya existentes, y se agregaron DTOs record de respuesta para usuarios, solicitudes, historial y errores. Esto deja la API más robusta, tipada e inmune al uso de Map<String, Object> como RequestBody, lo cual sí era una debilidad frente a las guías y frente a lo que pide el profesor.

Se añadieron mappers explícitos para convertir entre los objetos del dominio y los contratos REST. Esto permite que la capa web ya no dependa de exponer directamente las entidades del dominio y deja lista la estructura para cuando entre la persistencia JPA en la guía 10. Además, se agregó un GlobalExceptionHandler con @RestControllerAdvice para traducir excepciones de validación y del negocio a respuestas HTTP estándar, cubriendo el vacío que todavía existía en la guía 08.

Los controladores REST fueron actualizados para usar la nueva capa de casos de uso como dependencia principal, en lugar de seguir apoyándose en la capa antigua de ApplicationService. También se ajustaron las respuestas para usar DTOs de salida y se agregó soporte para endpoints de consulta por estado e historial auditable de solicitudes, dejando mejor cubiertos RF-06 y RF-07 dentro del alcance actual.

En la parte de persistencia temporal, se conectaron como beans Spring las implementaciones en memoria de la línea nueva, ubicadas en infrastructure.persistence.memory, de manera que la aplicación ya funciona apoyándose en los puertos de domain.repository. Esto es importante porque deja preparado el terreno para que en la guía 10 esas implementaciones en memoria sean reemplazadas por adaptadores JPA/H2 sin tocar el dominio ni la capa de aplicación.

Respecto a la deuda técnica, sigue existiendo una línea vieja compuesta por `application` e `infrastructure.persistence`, creada en una etapa anterior del proyecto. Esa línea no se elimina todavía para no afectar estabilidad ni romper el proyecto mientras se completa la guía 10. La línea nueva y correcta para continuar ya está implementada y alineada con la arquitectura de capas limpias, pero la línea vieja quedará temporalmente como legado hasta que en la guía 10 la persistencia real con H2/JPA quede funcionando sobre los puertos correctos. Una vez eso ocurra, ya será seguro retirar la línea anterior sin riesgo.

En resumen, lo que se hizo fue:

- consolidar la guía 09 sobre lo ya existente
- completar lo que faltaba de 07 y 08
- dejar la API usando casos de uso desacoplados
- agregar DTOs de respuesta, mappers y manejo global de excepciones
- conectar la línea nueva con Spring usando repositorios en memoria
- preparar el proyecto para que la guía 10 se implemente reemplazando memoria por H2/JPA, no reescribiendo todo

Para continuar con la guía 10 se debe tener en cuenta lo siguiente:

La ruta oficial a seguir es `domain`, `application.usecase` e `infrastructure`, dejando como referencia temporal la línea vieja hasta que la persistencia JPA quede funcionando.

Lo primero que debe tocarse es la capa de infraestructura de persistencia. Ahí debe crearse un nuevo paquete del estilo `infrastructure.persistence.jpa`, porque la guía 10 exige separar claramente el dominio puro de la materialización relacional. En esa capa deben aparecer las entidades JPA con `@Entity`, las relaciones necesarias y, cuando aplique, objetos embebibles con `@Embeddable`. Es importante que estas entidades JPA no sean las entidades del dominio, sino una representación persistente separada.

Después de eso, hay que crear los repositorios Spring Data con `JpaRepository`. Esos repositorios no reemplazan directamente a los puertos del dominio, sino que sirven como infraestructura de bajo nivel para acceder a H2. Encima de ellos debe construirse un adaptador de persistencia que implemente las interfaces que ya existen en `domain.repository`, especialmente `SolicitudRepository` y `UsuarioRepository`. Esa será la pieza que conecte los casos de uso con la base de datos real.

También hay que crear mappers de persistencia entre dominio y JPA. Así como ya existen mappers REST para DTOs, en la guía 10 deben existir mappers que conviertan:

- entidad de dominio -> entidad JPA
- entidad JPA -> entidad de dominio

Ese punto es clave para cumplir con la entrega, porque la rúbrica exige una frontera clara entre el dominio y la persistencia.

Una vez exista esa capa JPA, debe configurarse H2 en `application.properties`. Ahí toca agregar la URL de la base, driver, credenciales, configuración de consola H2 y opciones de Hibernate/JPA. En este momento el proyecto casi no tiene configuración ahí, así que esa parte está prácticamente virgen y será uno de los primeros pasos reales de la guía 10.

Cuando los adaptadores JPA ya estén listos, el siguiente paso es hacer que Spring inyecte esa nueva implementación en lugar de los repositorios en memoria de la línea nueva. Es decir, la API y los casos de uso deben seguir funcionando igual, pero guardando y consultando desde H2. Ese es justamente el objetivo de la guía: cambiar la implementación de persistencia sin romper dominio ni aplicación.

No conviene borrar de inmediato la línea vieja `application` e `infrastructure.persistence`, ni tampoco eliminar enseguida `infrastructure.persistence.memory`. Lo recomendable es:

- implementar primero JPA/H2 sobre `domain.repository`
- probar que controllers + use cases + adaptadores funcionan bien
- solo después retirar o dejar obsoletas las implementaciones viejas

En términos concretos, el camino sugerido para tu compañero sería:

1. Crear `infrastructure.persistence.jpa.entity`
2. Crear `infrastructure.persistence.jpa.repository`
3. Crear `infrastructure.persistence.jpa.mapper`
4. Crear adaptadores que implementen `domain.repository`
5. Configurar H2 y JPA en `application.properties`
6. Verificar que los controllers sigan funcionando sin tocar el dominio
7. Solo al final decidir qué clases viejas dejar como legado y cuáles retirar

Actualmente en el proyecto conviven dos líneas de implementación. Esto no significa que el proyecto esté mal, sino que fue creciendo por etapas y ahora ya hay una ruta correcta definida para continuar.

La **línea vieja** corresponde a la estructura que se había construido antes de consolidar la arquitectura por casos de uso y puertos. Esa línea está compuesta principalmente por:

- `application`
- `infrastructure/persistence`

Ahí se encuentran clases como:

- SolicitudApplicationService.java
- UsuarioApplicationService.java
- SolicitudRepository.java
- UsuarioRepository.java
- InMemorySolicitudRepository.java
- InMemoryUsuarioRepository.java

Esa línea fue útil para avanzar y dejó una base funcional, pero ya no es la que mejor representa la arquitectura que piden las guías y la segunda entrega. Por eso, **no debe seguirse expandiendo**. Sin embargo, tampoco debe eliminarse todavía, porque hasta que la guía 10 deje funcionando la persistencia JPA/H2 sobre la arquitectura nueva, conviene conservarla como respaldo temporal.

La **línea nueva**, que es la que ya quedó alineada y sobre la que debe seguir construyéndose el proyecto, está formada por:

- domain
- application/usecase
- domain/repository
- infrastructure/api
- infrastructure/api/dto
- infrastructure/api/mapper
- infrastructure/persistence/memory

Esta es la línea correcta porque:

- mantiene el dominio aislado de frameworks
- hace que la aplicación dependa de puertos, no de implementaciones concretas
- deja la API REST desacoplada mediante DTOs y mappers
- permite reemplazar la persistencia en memoria por JPA/H2 sin tocar el dominio ni los casos de uso

En otras palabras, **la línea vieja se conserva temporalmente pero no se expande más**, y **la línea nueva es la oficial para continuar con la guía 10 y con el resto de la segunda entrega**.

La transición correcta es esta:

- continuar desarrollando sobre la línea nueva
- implementar en la guía 10 la persistencia JPA/H2 sobre domain.repository

- una vez eso funcione, revisar qué partes de la línea vieja ya pueden retirarse sin afectar estabilidad

Ese es el criterio que mantiene el proyecto ordenado, coherente con la arquitectura limpia y seguro para seguir avanzando.

Una vez la guía 10 quede implementada correctamente y la persistencia real con H2/JPA funcione sobre la **línea nueva**, ya será seguro retirar la **línea vieja**. Esto significa que, cuando los controladores, los casos de uso y los repositorios estén trabajando completamente a través de `application.usecase`, `domain.repository` y los nuevos adaptadores JPA de `infrastructure.persistence.jpa`, la estructura anterior dejará de tener utilidad práctica y podrá eliminarse.

La condición para hacerlo no es simplemente “haber escrito la guía 10”, sino verificar que:

- los controladores ya dependan únicamente de la línea nueva
- los casos de uso ya utilicen los puertos de `domain.repository`
- la persistencia real esté funcionando con H2/JPA
- no exista ninguna dependencia activa hacia `application` ni hacia `infrastructure.persistence` de la línea vieja
- el proyecto compile y ejecute correctamente usando solo la arquitectura nueva

Cuando eso se cumpla, la línea vieja pasará a ser código redundante y ya podrá eliminarse sin afectar el funcionamiento del sistema. En ese momento, su eliminación no sería un riesgo, sino una limpieza arquitectónica necesaria para dejar el proyecto consistente con la segunda entrega y con la estructura hexagonal o limpia que exige la rúbrica.